

MHM Race Management

Central Server Deployment Guide

Medicine Hat Massacre -- SEASAR

Three deployment options covered in this guide:

A	Windows Host	LAN / same-network setup. Simplest, no extra hardware.
B	Raspberry Pi	Always-on low-cost host. Good for club-owned hardware.
C	Azure Web App	Public internet results page. Best for spectators anywhere.

All options use the same server/app.py FastAPI application. The client laptop connects by entering the server URL and API key in Settings.

System Architecture

The MHM server is a lightweight FastAPI application that acts as a central results hub during race day. The finish-line laptop (running MHM-Race.exe) pushes scored results to the server every 30 seconds over a pre-shared API key. Spectators, volunteers, and remote officials can view live standings at the server's public URL without needing access to the finish-line machine.

The server is intentionally minimal: it receives sync payloads, stores them in a local SQLite database, and serves a read-only results page. It has no SI reader, no chip scoring, and no admin UI -- all race management stays on the client laptop.

Components:

Finish-line laptop	Runs MHM-Race.exe. Reads SI chips, scores results, stores everything in mhm_local.db. Every 30 s it POSTs unsynced records to POST /api/sync on the server.
Central server	Receives the sync payload, upserts records into mhm_server.db, and serves GET / (live leaderboard) and GET /api/results (JSON feed).
Spectators / displays	Open the server URL in any browser. No login required. The page auto-refreshes every 30 s.
Authentication	A single pre-shared API key (MHM_API_KEY env var). The client sends it as the X-API-Key header. The server rejects any request with a wrong or missing key with HTTP 403.

Prerequisites (all options):

All three deployment methods require the same MHM server source files:

server/app.py, server/database.py, server/templates/
shared/models.py, shared/scoring.py
run_server.py, requirements-server.txt

requirements-server.txt installs: fastapi>=0.111, uvicorn[standard]>=0.29, jinja2>=3.1

Pick ONE of the three deployment options below. Option A (Windows) is the simplest for a local/LAN setup. Option B (Raspberry Pi) is good for a permanent low-cost host. Option C (Azure Web App) is the best choice for a public internet-facing results page.

Option A -- Windows Host (LAN / same network)

Best for: club events where spectators are on the same Wi-Fi network as the server, or where the server is a second laptop connected to a local router.

The server URL will be something like `http://192.168.1.50:8000` -- only reachable from devices on the same network.

1 Install Python 3.11+

Download from python.org and install. Tick 'Add Python to PATH' during setup.

Verify: open a Command Prompt and run: `python --version`

2 Copy the server files

Copy the following folders/files from the MHM project to the server machine (USB stick, shared drive, or git clone):

server/

shared/

run_server.py

requirements-server.txt

3 Install dependencies

Open a Command Prompt in the folder containing `run_server.py` and run:

```
pip install -r requirements-server.txt
```

4 Set the API key

In the same Command Prompt, set the environment variable before starting:

```
set MHM_API_KEY=your-secret-key-here
```

Choose a strong random key (e.g. 32+ random characters). Write it down -- you will enter the same key in the client app's Settings page.

5 Start the server

Run: `python run_server.py`

You should see: INFO: Uvicorn running on `http://0.0.0.0:8000`

Open `http://localhost:8000` in a browser to confirm the results page loads.

6 Find the server's IP address

In a second Command Prompt run: `ipconfig`

Look for the IPv4 Address under your active network adapter (e.g. 192.168.1.50). This is the URL clients and spectators will use: `http://192.168.1.50:8000`

7 Open Windows Firewall

If other devices cannot reach the server, allow port 8000 inbound:

1. Open Windows Defender Firewall with Advanced Security
2. Inbound Rules -> New Rule -> Port -> TCP -> 8000
3. Allow the connection, apply to all profiles, name it 'MHM Server'

8 Run as a background service (optional)

To keep the server running after closing the Command Prompt, use NSSM (Non-Sucking Service Manager). Download from nssm.cc, then:

```
nssm install MHMServer python C:\path\to\run_server.py  
nssm set MHMServer AppEnvironmentExtra MHM_API_KEY=your-secret-key  
nssm start MHMServer
```

The server will now start automatically with Windows.

Option B -- Raspberry Pi (always-on local/LAN host)

Best for: clubs that want a permanent low-cost server that stays on all event weekend. A Pi 3B+ or newer is more than capable. If the Pi has a public IP (e.g. via a 4G dongle or port-forwarded router), it can serve spectators over the internet too.

1 Flash Raspberry Pi OS Lite

Download Raspberry Pi Imager from [raspberrypi.com](https://www.raspberrypi.com). Flash 'Raspberry Pi OS Lite (64-bit)' to a microSD card. In Advanced Options, enable SSH and set a hostname (e.g. mhm-server.local), username, and password before flashing.

2 Boot and connect

Insert the SD card, power on the Pi, and wait ~60 s. SSH in from your laptop:

```
ssh pi@mhm-server.local
(Replace 'pi' with the username you set.)
```

3 Update and install Python

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv git
```

Python 3.11+ is included in recent Raspberry Pi OS. Verify: `python3 --version`

4 Copy or clone the server files

Option 1 -- git clone (recommended if your repo is on GitHub):

```
git clone https://github.com/YOUR_ORG/mhm.git ~/mhm
```

Option 2 -- SCP from your laptop:

```
scp -r ./server ./shared run_server.py requirements-server.txt \
pi@mhm-server.local:~/mhm/
```

5 Create a virtual environment and install deps

```
cd ~/mhm
```

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-server.txt
```

6 Set the API key and test

```
export MHM_API_KEY=your-secret-key-here
python run_server.py
```

Visit `http://mhm-server.local:8000` from another device on the same network to confirm the page loads. Press Ctrl+C to stop.

7 Run as a systemd service

Create `/etc/systemd/system/mhm-server.service`:

```
[Unit]
Description=MHM Results Server
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/mhm
Environment=MHM_API_KEY=your-secret-key-here
```

```
ExecStart=/home/pi/mhm/.venv/bin/uvicorn server.app:app \
  --host 0.0.0.0 --port 8000
Restart=always
```

```
[Install]
WantedBy=multi-user.target
```

Then enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable mhm-server
sudo systemctl start mhm-server
sudo systemctl status mhm-server
```

8 Reserve a static IP (optional but recommended)

Log into your router's admin page, find the Pi's MAC address, and assign it a reserved (static) DHCP lease so its IP never changes. Alternatively edit `/etc/dhcpd.conf` on the Pi to set a static IP directly.

Option C -- Azure Web App (public internet)

Best for: public-facing results where spectators anywhere can view standings via a real URL (e.g. <https://mhm-results.azurewebsites.net>). Requires an Azure account. The Free or Basic tier is sufficient for race-day traffic.

Azure Web Apps run the server via the Procfile command:

```
web: uvicorn server.app:app --host 0.0.0.0 --port $PORT
```

1 Prerequisites

- Azure account (free.azure.com)
- Azure CLI installed: docs.microsoft.com/cli/azure/install-azure-cli
- Git repository containing the MHM server files (GitHub, Azure DevOps, etc.)
- The Procfile and requirements-server.txt must be in the repo root

2 Log in to Azure CLI

```
az login
```

A browser window opens. Sign in with your Azure account.

3 Create a Resource Group

```
az group create --name mhm-rg --location canadacentral
```

Choose a location close to your event (canadacentral, eastus, westus2, etc.).

4 Create an App Service Plan (Free tier)

```
az appservice plan create \
  --name mhm-plan \
  --resource-group mhm-rg \
  --sku F1 \
  --is-linux
```

F1 is the free tier. Upgrade to B1 (~\$13/month) for a custom domain and always-on (prevents cold starts). B1 is recommended for race day.

5 Create the Web App

```
az webapp create \
  --name mhm-results \
  --resource-group mhm-rg \
  --plan mhm-plan \
  --runtime "PYTHON:3.12"
```

Your app URL will be: <https://mhm-results.azurewebsites.net>

Choose a unique name -- it must be globally unique on azurewebsites.net.

6 Set the API key as an App Setting

```
az webapp config appsettings set \
  --name mhm-results \
  --resource-group mhm-rg \
  --settings MHM_API_KEY=your-secret-key-here
```

App Settings are injected as environment variables at runtime. Never put the API key in source code or commit it to git.

7 Set the startup command

```
az webapp config set \
  --name mhm-results \
  --resource-group mhm-rg \
  --startup-file "uvicorn server.app:app --host 0.0.0.0 --port 8000"
```

Alternatively, Azure will auto-detect the Procfile if it is in the repo root.

8 Deploy via Git (local git method)

```
# Configure deployment credentials
az webapp deployment user set \
  --user-name mhmadmin --password YourDeployPassword1!

# Get the git remote URL
az webapp deployment source config-local-git \
  --name mhm-results --resource-group mhm-rg

# Push your code
git remote add azure <URL from above command>
git push azure main
```

Azure will install dependencies from requirements-server.txt and start the app.

9 Alternatively: deploy via GitHub Actions

In the Azure Portal, go to your Web App -> Deployment Center -> GitHub. Authorize Azure, select your repo and branch, and click Save. Azure creates a GitHub Actions workflow that deploys automatically on every push to main.

10 Verify the deployment

Visit <https://mhm-results.azurewebsites.net> in a browser. You should see the MHM results page (empty until the client syncs data).

Check logs if something is wrong:

```
az webapp log tail --name mhm-results --resource-group mhm-rg
```

11 Persistent database note

By default, Azure Web Apps use ephemeral local storage -- the SQLite database is lost on each restart or redeploy.

Fix: mount an Azure Files share as persistent storage:

```
az webapp config storage-account add \
  --name mhm-results --resource-group mhm-rg \
  --custom-id mhm-data \
  --storage-type AzureFiles \
  --account-name <storage-account> \
  --share-name mhm-db \
  --mount-path /app/server
```

Then update server/database.py to point DB_PATH to /app/server/mhm_server.db when running on Azure (check for WEBSITE_SITE_NAME env var).

12 Custom domain (optional)

Upgrade the App Service Plan to B1 first (free tier doesn't support custom domains).

Then in the Azure Portal -> your Web App -> Custom Domains -> Add Custom Domain. Point your DNS provider's CNAME record to mhm-results.azurewebsites.net. Azure can also provision a free managed TLS certificate automatically.

Connecting the Client to the Server

Once the server is running, configure the finish-line laptop to push data to it. All settings are in the MHM client's Settings page.

1 Open Settings on the finish-line laptop

Launch MHM-Race.exe and click Settings in the top navigation.

2 Enter the Server URL

In the 'Server URL' field, enter the full URL of the server:

Windows LAN: `http://192.168.1.50:8000`

Raspberry Pi: `http://mhm-server.local:8000`

Azure: `https://mhm-results.azurewebsites.net`

Do NOT include a trailing slash.

3 Enter the API Key

In the 'API Key' field, enter the same key you set in `MHM_API_KEY` on the server. The client sends this as the `X-API-Key` header with every sync request.

4 Set the sync interval

The default is 30 seconds. Increase to 60 s on a slow or unreliable connection. Decrease to 10 s if you want near-real-time updates on the public display.

5 Save and confirm

Click Save Settings. The Dashboard will show the sync status:

Green dot = connected, syncing to <URL> every N s

No dot / offline = URL is blank or server unreachable

After the first chip is read and saved, visit the server URL in a browser -- the team should appear in the results within one sync interval.

6 Troubleshoot sync failures

If the Dashboard shows errors or results don't appear on the server:

- Check the server URL has no trailing slash and is exactly correct
- Confirm the API key matches on both sides (case-sensitive)
- Check the server is running (visit its URL directly in a browser)
- On Windows LAN: confirm the firewall allows port 8000
- On Azure: check logs with: `az webapp log tail --name mhm-results --resource-group mhm-rg`
- Look in `mhm.log` (same folder as the .exe) for 'Sync failed' warning lines

Security & Production Checklist

1 API key strength

Use a randomly generated key of at least 32 characters. Example generation:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Never use the default 'changeme' key in production.

2 HTTPS

The sync POST includes the API key in a plain HTTP header. On a local LAN this is acceptable. For internet-facing deployments (Azure, public Pi), always use HTTPS. Azure Web Apps provide HTTPS automatically. For a Pi exposed to the internet, put Nginx + Certbot (Let's Encrypt) in front of uvicorn.

3 Nginx reverse proxy for Pi (HTTPS)

```
sudo apt install -y nginx certbot python3-certbot-nginx
# Create /etc/nginx/sites-available/mhm with:
#   server { listen 80; server_name results.yourdomain.com;
#       location / { proxy_pass http://127.0.0.1:8000; } }
sudo ln -s /etc/nginx/sites-available/mhm /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d results.yourdomain.com
```

4 Database backups

The server database (mhm_server.db) is a SQLite file. Back it up by copying it off the host at the end of each day. On Linux:

```
sqlite3 mhm_server.db '.backup /backup/mhm_server_backup.db'
```

The client (mhm_local.db) is the authoritative source -- if the server database is lost, the client can re-sync all data by resetting the synced flags (contact developer for the reset script).

5 Keeping the server up to date

If the shared/ scoring logic changes between events, redeploy the server with the updated files before the event. The server and client must run the same version of shared/models.py and shared/scoring.py to score correctly.

6 Restrict public write access

The /api/sync endpoint is protected by the API key. The public GET / and GET /api/results endpoints are intentionally unauthenticated -- anyone with the URL can view results. If you want to restrict viewing, add HTTP Basic Auth in Nginx or as FastAPI middleware before the event.

Quick-Reference: Deployment Checklist

1 Server setup

- ☐ Choose deployment option (Windows / Pi / Azure)
- ☐ Generate a strong API key (python -c "import secrets; print(secrets.token_urlsafe(32))")
- ☐ Deploy server files and install requirements-server.txt
- ☐ Set MHM_API_KEY environment variable / App Setting
- ☐ Start the server and confirm the results page loads in a browser
- ☐ (Azure) Attach persistent Azure Files storage for the database
- ☐ (Internet) Configure HTTPS

2 Client configuration

- ☐ Open MHM-Race.exe -> Settings
- ☐ Enter server URL (no trailing slash)
- ☐ Enter API key (must match server exactly)
- ☐ Set sync interval (default 30 s)
- ☐ Click Save Settings
- ☐ Read a test chip and confirm the result appears on the server within 30 s

3 On race day

- ☐ Confirm server is running before teams start
- ☐ Display server URL on a screen for spectators
- ☐ Monitor client Dashboard -- sync status dot should stay green
- ☐ If sync fails: check mhm.log and server logs
- ☐ Back up mhm_server.db at end of day